

KINESYS

Sistema de Gestión de Clínica de Fisioterapia

Proyecto final - primera entrega

Documento diligenciado según plantilla de pruebas

Módulos evaluados:

Registro/Gestión de Pacientes

Registro/Gestión de Terapeutas

Gestión de Terapias y Tratamientos

Estudiantes:

Eckard Antony Rodriguez Gutierrez

Luis Alejandro Narvaez Teran

Docente: Maribel Romero Mestre

Universidad Popular del Cesar - Sede Sabanas

Ingeniería de Sistemas

Valledupar - Cesar

2026

CONTENIDO

PRIMERA PARTE

Descripción del sistema

Identificación del problema

Descripción detallada del sistema o aplicación

Modelo de requerimientos

Modelo de casos de uso

Modelo de diseño del sistema

Producto del software

SEGUNDA PARTE

Pruebas del software

Introducción

Planificación de las pruebas

Pruebas unitarias

Pruebas de integración

Pruebas de sistemas

Pruebas de aceptación

Conclusiones

PRIMERA PARTE

DESCRIPCIÓN DEL SISTEMA

IDENTIFICACIÓN DEL PROBLEMA

En la clínica de fisioterapia se requiere controlar información crítica de pacientes, terapeutas y tratamientos. Cuando estos procesos se hacen de forma manual se presentan registros incompletos, duplicidad de documentos, dificultad para diferenciar pacientes y terapeutas, errores en la asignación de tratamientos y poca trazabilidad de equipos requeridos.

Para esta entrega se delimita el trabajo a tres módulos que ya venían desarrollados en el PDF inicial: Registro/Gestión de Pacientes, Registro/Gestión de Terapeutas y Gestión de Terapias/Tratamientos.

DESCRIPCIÓN DETALLADA DEL SISTEMA O APLICACIÓN

KineSys es una aplicación web construida con React y Vite en el frontend, y ASP.NET Core Web API con Entity Framework Core y SQL Server en el backend. La aplicación automatiza la administración de pacientes, terapeutas y tratamientos mediante formularios, listados, servicios de comunicación con API y validaciones funcionales.

Los procesos automatizados en el alcance de este documento son: creación y filtrado de pacientes por rol 1, creación y filtrado de terapeutas por rol 2, mantenimiento de tratamientos, relación de tratamientos con especialidades y equipos, validación de datos de entrada, control de errores del API y pruebas automatizadas de servicios.

MODELO DE REQUERIMIENTOS

Descripción Requisitos Funcionales

ID	Módulo	Requisito funcional
RF-PAC-01	Pacientes	Registrar paciente con datos personales, documento, contacto, EPS/seguro y rol Paciente.
RF-PAC-02	Pacientes	Listar únicamente personas con rol = 1.
RF-PAC-03	Pacientes	Consultar, actualizar y eliminar pacientes.
RF-TER-01	Terapeutas	Registrar terapeuta con datos de acceso, personales, contacto y profesionales.
RF-TER-02	Terapeutas	Listar únicamente personas con rol = 2 y obtener terapeutas activos.
RF-TER-03	Terapeutas	Actualizar terapeuta preservando rol Terapeuta.
RF-TRA-01	Tratamientos	Registrar tratamiento con nombre, especialidad, duración, costo, sesiones y frecuencia.
RF-TRA-02	Tratamientos	Listar, consultar, actualizar y eliminar tratamientos.
RF-TRA-03	Tratamientos	Agregar y remover equipos requeridos del tratamiento.

Descripción Requisitos No Funcionales

ID	Atributo	Requisito no funcional
RNF-01	Mantenibilidad	Separar interfaz, servicios y API client por módulo.
RNF-02	Seguridad	Consumir endpoints mediante apiCliente con token JWT cuando exista sesión.
RNF-03	Integridad	Diferenciar paciente y terapeuta por rol y validar especialidad de tratamiento.
RNF-04	Usabilidad	Presentar formularios con campos requeridos, mensajes de error y estados de carga.
RNF-05	Testabilidad	Contar con pruebas automatizadas en Jest/Vitest para servicios críticos.

MODELO DE CASOS DE USO

Diagrama de caso de uso

Actor	Casos de uso
Administrador	Registrar paciente, consultar paciente, actualizar paciente, eliminar paciente.
Administrador	Registrar terapeuta, consultar terapeuta, actualizar terapeuta, eliminar terapeuta, listar terapeutas activos.
Administrador	Registrar tratamiento, consultar tratamiento, actualizar tratamiento, eliminar tratamiento, agregar/remover equipos.
Sistema	Validar rol, enviar datos a API, manejar errores 409/500, actualizar listados.

Descripción de caso de uso

ID Caso Uso	Nombre	Actor	Flujo principal	Resultado
CU-PAC-01	Registrar Paciente	Administrador	Abrir formulario, ingresar datos válidos, guardar.	Paciente creado con rol 1.
CU-TER-01	Registrar Terapeuta	Administrador	Abrir formulario, ingresar acceso y datos profesionales, guardar.	Terapeuta creado con rol 2.
CU-TRA-01	Registrar Tratamiento	Administrador	Abrir formulario, seleccionar especialidad, definir duración/costo/equipos, guardar.	Tratamiento creado.

MODELO DE DISEÑO DEL SISTEMA

Modelo solicitado	Contenido adelantado
Diagrama de clases detallado	Persona hereda a Paciente y Terapeuta. Tratamiento se relaciona con especialidades, equipos y servicios.
Diagramas de secuencias	Formulario React -> service.js -> apiCliente -> API .NET -> SQL Server -> respuesta.
Diagrama entidad relación	Paciente, Terapeuta y Tratamiento se persisten como entidades del contexto ClinicaFisioterapiaBD.
Diagrama de componentes	Pages/Formularios, services, api, backend controllers, EF Core, SQL Server.

PRODUCTO DEL SOFTWARE

Pantallazos de la aplicación correspondientes a los tres módulos trabajados:

Registrar Nuevo Paciente



Información Personal

Tipo Documento *	Número Documento *	Género
Cédula de Ciudadanía	1234567890	Seleccionar
Nombres *		Apellidos *
Fecha Nacimiento	Email	Celular
dd/mm/aaaa		

Contacto y Ubicación

Teléfono	Ciudad	Departamento
Dirección Completa		

EPS

[+ Agregar EPS](#)

Seleccionar EPS
Sin EPS

Seguro Médico

[+ Agregar Seguro](#)

Registrar Nuevo Terapeuta



Información de Acceso



Usuario *

Nombre de usuario único

El nombre de usuario debe ser único en el sistema



Contraseña *

Contraseña del usuario

La contraseña debe tener al menos 6 caracteres

Información Personal

Tipo Documento *

Seleccionar

Número Documento *

1234567890

Género

Seleccionar

Nombres *

Apellidos *

Fecha Nacimiento

dd/mm/aaaa



Email

Celular

Contacto y Ubicación

Teléfono

Ciudad

Departamento

Dirección Completa

Pantallazo módulo Registro/Gestión de Terapeutas

Registrar Nuevo Tratamiento



Información Básica

Nombre del Tratamiento *

Ej: Terapia de Rehabilitación

Especialidad *

Selecciona una especialidad

* Campo requerido. Selecciona la especialidad a la que pertenece este tratamiento.

Descripción

Descripción detallada del tratamiento...

Configuración de Sesiones

Duración (minutos)

30

Sesiones Recomendadas

1

Frecuencia

Semanal

Equipos Utilizados

Seleccionar Equipo

Cargando equipos...

Cantidad

1

+ Agregar



Pantallazo módulo Gestión de Terapias y Tratamientos

Elemento	Acceso
Frontend local	cd kinesys; npm run dev; http://127.0.0.1:5173
Pruebas	cd kinesys; npm run test / npm run test:jest / npm run test:coverage
Backend local	cd api/ApiPrueba; dotnet run
Link público	Pendiente de despliegue.

SEGUNDA PARTE

PRUEBAS DEL SOFTWARE

INTRODUCCIÓN

La sección de pruebas documenta el proceso de validación de los tres módulos seleccionados. Se aplican pruebas unitarias, integración, sistema y aceptación, siguiendo las tablas y apartados solicitados por la plantilla.

PLANIFICACIÓN DE LAS PRUEBAS

Objetivos

Objetivo General: verificar y validar que los módulos de Pacientes, Terapeutas y Terapias/Tratamientos funcionen conforme a los requisitos definidos, con datos válidos y no válidos, control de errores y evidencia automatizada.

- Verificar que el módulo Pacientes registre, liste, consulte, actualice y elimine pacientes usando rol 1.
- Validar que el módulo Terapeutas registre, liste, consulte, actualice y filtre terapeutas usando rol 2.
- Comprobar que el módulo Tratamientos registre tratamientos válidos y gestione equipos asociados.
- Ejecutar pruebas unitarias con Vitest/Jest y evaluar resultados.

Alcance

Nombre de la aplicación a probar	KineSys - Sistema de Gestión de Clínica de Fisioterapia
Módulos a ser probados	Objetivos de las pruebas
Módulo 1: Registro/Gestión de Pacientes	Validar datos personales, documento, rol Paciente, creación, consulta, actualización y eliminación.
Módulo 2: Registro/Gestión de Terapeutas	Validar datos de acceso, personales, profesionales, rol Terapeuta, creación, consulta, actualización y activos.
Módulo 3: Gestión de Terapias y Tratamientos	Validar nombre, especialidad, duración, costo, sesiones, frecuencia, equipos y CRUD de tratamientos.

Estrategias De Pruebas

Niveles, tipos y métodos de prueba			
Niveles de pruebas	Técnicas de pruebas	Modelos/estrategias/tipos	Herramientas de pruebas
Pruebas unitarias	Caja negra y caja blanca	Clases de equivalencia, valores límites, camino básico	Vitest, Jest, Testing Library, mocks de API
Pruebas de integración	Caja negra	Incremental y basada en hilos	Vitest/Jest, revisión de flujo Formulario -> Servicio -> API
Pruebas de sistemas	Caja negra	Rendimiento, seguridad, usabilidad y portabilidad	Vite build, revisión funcional, evidencia local
Pruebas de aceptación	Caja negra	Casos de usuario final por módulo	Manual; pendiente Selenium/Katalon

Ambiente De Pruebas

EQUIPO	ESPECIFICACIONES TECNICAS
Equipo de desarrollo	Windows 11 Home Single Language 64 bits; Intel Core i5-1235U; 10 núcleos; 12 procesadores lógicos; 15.7 GB RAM.
Repositorio local	C:\Users\carde\Documents\ProgramacionMovil\KineSys

SOFTWARE	VERSION
Node.js	v24.14.0
npm	11.9.0
React	19.1.1
Vite	7.x
Vitest	4.1.5
Jest	30.3.0
ASP.NET Core	net8.0
SQL Server	Configurado por docker-compose

PRUEBAS UNITARIAS

[Los siguientes pasos se repiten por cada componente a probar]

MÓDULO 1: REGISTRO/GESTIÓN DE PACIENTES

Análisis De Las Pruebas

Nombre del Módulo: Registro/Gestión de Pacientes

KineSys

Iniciar Sesión

Registrar Nuevo Paciente

Información Personal

Tipo Documento *

Número Documento *

Género

Cédula de Ciudadanía

1234567890

Seleccionar

Nombres *

Apellidos *

Fecha Nacimiento

Email

Celular

dd/mm/aaaa

Contacto y Ubicación

Teléfono

Ciudad

Departamento

Dirección Completa

EPS

+ Agregar EPS

Seleccionar EPS

Sin EPS

Seguro Médico

+ Agregar Seguro

Interfaz de entrada - Pacientes

Análisis de pruebas clases de equivalencias

Condiciones de Entrada	Clases de equivalencia válida	Clases de equivalencia no válida
Nombres	CE-PAC-01: 2 a 50 caracteres, solo letras y espacios	CE-PAC-02: vacío; CE-PAC-03: mayor a 50; CE-PAC-04: contiene números
Apellidos	CE-PAC-05: 2 a 50 caracteres, solo letras y espacios	CE-PAC-06: vacío; CE-PAC-07: mayor a 50; CE-PAC-08: contiene números
Tipo Documento	CE-PAC-09: CC; CE-PAC-10: TI; CE-PAC-11: CE; CE-PAC-12: PA	CE-PAC-13: valor no permitido o sin selección
Documento Identidad	CE-PAC-14: 6 a 15 dígitos	CE-PAC-15: menor a 6; CE-PAC-16: mayor a 15; CE-PAC-17: contiene letras; CE-PAC-18: vacío
Correo	CE-PAC-19: formato usuario@dominio.com	CE-PAC-20: sin @; CE-PAC-21: sin dominio; CE-PAC-22: vacío si es obligatorio
Teléfono/Celular	CE-PAC-23: 10 dígitos	CE-PAC-24: menor a 10; CE-PAC-25: mayor a 10; CE-PAC-26: contiene letras
Fecha nacimiento	CE-PAC-27: fecha válida pasada	CE-PAC-28: fecha futura; CE-PAC-29: formato inválido
EPS/Afiliación	CE-PAC-30: EPS existente o Sin EPS	CE-PAC-31: EPS inexistente; CE-PAC-32: afiliación incompleta cuando se selecciona EPS

Casos de pruebas

Nombres	Apellidos	Tipo Doc	Documento	Email	Teléfono	Clases cubiertas	Resultado esperado
Ana	Pérez	CC	12345678	ana@mail.com	3001234567	CE-PAC-01,05,09,14,19,23	Registro exitoso
	Pérez	CC	12345678	ana@mail.com	3001234567	CE-PAC-02	Error: nombre requerido
Ana		CC	12345678	ana@mail.com	3001234567	CE-PAC-06	Error: apellido requerido
Ana l	Pérez	CC	12345678	ana@mail.com	3001234567	CE-PAC-04	Error: nombre inválido
Ana	Pérez	CC	12345	ana@mail.com	3001234567	CE-PAC-15	Error: documento corto
Ana	Pérez	CC	12345678	anamail.com	3001234567	CE-PAC-20	Error: correo inválido

MÓDULO 2: REGISTRO/GESTIÓN DE TERAPEUTAS

Análisis De Las Pruebas

Nombre del Módulo: Registro/Gestión de Terapeutas

Registrar Nuevo Terapeuta



Información de Acceso



Usuario *

Nombre de usuario único

El nombre de usuario debe ser único en el sistema



Contraseña *

Contraseña del usuario

La contraseña debe tener al menos 6 caracteres

Información Personal

Tipo Documento *

Seleccionar

Número Documento *

1234567890

Género

Seleccionar

Nombres *

Apellidos *

Fecha Nacimiento

dd/mm/aaaa



Email

Celular

Contacto y Ubicación

Teléfono

Ciudad

Departamento

Dirección Completa

Interfaz de entrada - Terapeutas

Análisis de pruebas clases de equivalencias

Condiciones de Entrada	Clases de equivalencia válida	Clases de equivalencia no válida
Usuario	CE-TER-01: 4 a 20 caracteres alfanuméricos	CE-TER-02: vacío; CE-TER-03: menor a 4; CE-TER-04: mayor a 20; CE-TER-05: caracteres especiales
Contraseña	CE-TER-06: mínimo 6 caracteres en creación	CE-TER-07: vacía en creación; CE-TER-08: menor a 6
Tipo Documento	CE-TER-09: CC; CE-TER-10: CE; CE-TER-11: TI; CE-TER-12: PA	CE-TER-13: sin selección
Documento Identidad	CE-TER-14: 6 a 15 dígitos	CE-TER-15: menor a 6; CE-TER-16: mayor a 15; CE-TER-17: caracteres inválidos
Nombres/Apellidos	CE-TER-18: 2 a 50 caracteres	CE-TER-19: menor a 2; CE-TER-20: mayor a 50; CE-TER-21: vacío
Licencia profesional	CE-TER-22: texto no vacío	CE-TER-23: vacío; CE-TER-24: duplicado
Título académico	CE-TER-25: texto no vacío	CE-TER-26: vacío; CE-TER-27: demasiado corto
Años experiencia	CE-TER-28: 0 a 99	CE-TER-29: negativo; CE-TER-30: mayor a 99; CE-TER-31: texto

Casos de pruebas

Condiciones de Entrada	Clases de equivalencia válida	Clases de equivalencia no válida
Nombre	CE-TRA-01: texto no vacío	CE-TRA-02: vacío; CE-TRA-03: solo espacios
Especialidad	CE-TRA-04: ID existente y diferente de 0	CE-TRA-05: 0; CE-TRA-06: inexistente
Duración	CE-TRA-07: 5 a 240 minutos	CE-TRA-08: menor a 5; CE-TRA-09: mayor a 240; CE-TRA-10: no numérico
Costo base	CE-TRA-11: mayor o igual a 0	CE-TRA-12: negativo; CE-TRA-13: no numérico
Sesiones recomendadas	CE-TRA-14: 1 a 50	CE-TRA-15: menor a 1; CE-TRA-16: mayor a 50
Frecuencia	CE-TRA-17: Diaria, Semanal, Quincenal o Mensual	CE-TRA-18: valor diferente
Equipo requerido	CE-TRA-19: equipo existente y cantidad >= 1	CE-TRA-20: sin equipo; CE-TRA-21: cantidad 0; CE-TRA-22: cantidad mayor a disponibilidad sin confirmación

Casos de pruebas

Nombre	Especialidad	Duración	Costo	Sesiones	Frecuencia	Clases cubiertas	Resultado esperado
Terapia fisica	1	60	150000	10	Semanal	CE-TRA-01,04,07,11,14,17	Registro exitoso
	1	60	150000	10	Semanal	CE-TRA-02	Error: nombre requerido
	1	60	150000	10	Semanal	CE-TRA-03	Error: nombre requerido
Rehabilitación	0	60	150000	10	Semanal	CE-TRA-05	Error: especialidad requerida
Masaje	1	4	80000	10	Semanal	CE-TRA-08	Error: duración fuera de límite
Electroterapia	1	241	80000	10	Semanal	CE-TRA-09	Error: duración fuera de límite
Terapia manual	1	60	-1	10	Semanal	CE-TRA-12	Error: costo inválido
Terapia manual	1	60	150000	51	Semanal	CE-TRA-16	Error: sesiones inválidas

Casos De Pruebas Por Valores Límites

MÓDULO	CAMPO	Datos de entrada	Escenario	Resultado
Paciente	Documento	12345	Límite inferior -1	Error
Paciente	Documento	123456	Límite inferior exacto	OK
Paciente	Documento	123456789012345	Límite superior exacto	OK
Paciente	Documento	1234567890123456	Límite superior +1	Error
Paciente	Teléfono	300123456	Límite inferior -1	Error
Paciente	Teléfono	3001234567	Límite exacto	OK
Terapeuta	Usuario	abc	Límite inferior -1	Error
Terapeuta	Usuario	abcd	Límite inferior exacto	OK
Terapeuta	Contraseña	12345	Límite inferior -1	Error
Terapeuta	Contraseña	123456	Límite inferior exacto	OK
Terapeuta	Años experiencia	-1	Límite inferior -1	Error
Terapeuta	Años experiencia	0	Límite inferior exacto	OK
Tratamiento	Duración	4	Límite inferior -1	Error
Tratamiento	Duración	5	Límite inferior exacto	OK
Tratamiento	Duración	240	Límite superior exacto	OK
Tratamiento	Duración	241	Límite superior +1	Error
Tratamiento	Sesiones	0	Límite inferior -1	Error
Tratamiento	Sesiones	1	Límite inferior exacto	OK
Tratamiento	Costo	-1	Límite inferior -1	Error
Tratamiento	Costo	0	Límite inferior exacto	OK

Pruebas Del Camino Básico

Caso de uso a validar: registro/actualización de cada uno de los tres módulos trabajados.

```
Fragmentos de métodos validados:

crearPaciente(data):
  datosCompleto = { ...data, user: documentoIdentidad, password: documentoIdentidad, activo: true,
fechaRegistro }
  pacientesApi.create('/Register', datosCompleto)

crearTerapeuta(data):
  datosCompleto = { ...data, activo: true, fechaRegistro, rol: 2 }
  terapeutasApi.create('/Register', datosCompleto)

crearTratamiento(data):
  tratamientosApi.create('', tratamientoData)
  si error 409: "Ya existe un tratamiento con ese nombre"
```

CAMINO	DATOS ENTRADA	ESCENARIO
PAC-C1	Paciente válido: Ana Pérez, CC 12345678	Paciente creado correctamente.
PAC-C2	API retorna persona con rol 2	obtenerPaciente lanza error: no es un paciente.
PAC-C3	API responde 409	Mensaje de documento duplicado.
TER-C1	Terapeuta válido con licencia y título	Terapeuta creado con rol 2.
TER-C2	Actualizar terapeuta sin rol en data	Servicio preserva rol 2.
TER-C3	API retorna persona con rol 1	obtenerTerapeuta lanza error: no es un terapeuta.
TRA-C1	Tratamiento válido con especialidad 1	Tratamiento creado correctamente.
TRA-C2	Especialidad 0	Formulario bloquea y muestra error.
TRA-C3	API responde 409	Mensaje de tratamiento duplicado.

Diseño Casos De Pruebas

S#	Prerrequisitos	S#	Datos de pruebas
1	Aplicación frontend en ejecución	1	Paciente con documento 12345678
2	Servicios API mockeados o backend disponible	2	Terapeuta con usuario ter123 y licencia FT-001
3	Usuario administrador con acceso al módulo	3	Tratamiento Terapia física con especialidad 1
4	Vitest/Jest instalados	4	Casos válidos y no válidos por equivalencia

PASO	Detalles del Paso	Resultados Esperados	Resultados Actuales	Estado
1	Ejecutar prueba de pacientes	12 casos aprobados	12 casos aprobados	Ejecutado
2	Ejecutar prueba de terapeutas	12 casos aprobados	12 casos aprobados	Ejecutado
3	Ejecutar prueba de tratamientos	9 casos aprobados	9 casos aprobados	Ejecutado
4	Ejecutar suite completa frontend	107 casos aprobados	107 casos aprobados	Ejecutado

Ejecución Y Evaluación De Las Pruebas

Prueba		
Caso de prueba	Imagen de la prueba	Resultado
npm run test -- src/services/pacientesService.test.js	Evidencia: artifacts/pruebas/pacientes-vitest.txt	Test Files: 1 passed. Tests: 12 passed.
Observaciones Técnicas/funcionales		Valida filtrado por rol 1, creación con user/password por documento, actualización, eliminación y errores.
Concepto Final		Aprobado

Prueba		
Caso de prueba	Imagen de la prueba	Resultado
npm run test -- src/services/terapeutasService.test.js	Evidencia: artifacts/pruebas/terapeutas-vitest.txt	Test Files: 1 passed. Tests: 12 passed.
Observaciones Técnicas/funcionales		Valida filtrado por rol 2, creación activa, actualización preservando rol, eliminación y activos.
Concepto Final		Aprobado

Prueba		
Caso de prueba	Imagen de la prueba	Resultado
npm run test -- src/services/tratamientosService.test.js	Evidencia: artifacts/pruebas/tratamientos-vitest.txt	Test Files: 1 passed. Tests: 9 passed.
Observaciones Técnicas/funcionales		Valida CRUD de tratamientos, error 409 y rutas anidadas de equipos.
Concepto Final		Aprobado

PRUEBAS DE INTEGRACIÓN

ESTRATEGIA DE PRUEBAS INCREMENTALES

Análisis De Las Pruebas

Esquema de integración de los componentes de la aplicación: Interfaz React -> service.js -> apiCliente -> API .NET -> Base de datos.

Tipo de integración	Descripción
Integración Incremental Ascendente	Se validan primero los servicios pacientesService, terapeutasService y tratamientosService; luego se conectan con formularios/listados.
Integración Incremental Descendente	Se parte de los formularios de usuario y se verifica que construyan datos compatibles con los servicios.

Diseño De Los Casos De Pruebas

ID	Integración	Datos	Resultado esperado	Estado
INT-PAC-01	PatientForm -> pacientesService -> pacientesApi	Datos válidos de paciente	Payload incluye user/password igual al documento, activo true y fechaRegistro.	Ejecutado parcialmente en servicio
INT-TER-01	TerapeutaForm -> terapeutasService -> terapeutasApi	Datos válidos de terapeuta	Payload incluye rol 2 y datos profesionales.	Ejecutado parcialmente en servicio
INT-TRA-01	TratamientoForm -> tratamientosService -> tratamientosApi	Datos válidos de tratamiento	Payload enviado al endpoint base de tratamientos.	Ejecutado parcialmente en servicio
INT-TRA-02	TratamientoForm -> equipos	Equipo y cantidad	Ruta /{idTratamiento}/equipos para agregar y remover equipos.	Ejecutado en servicio

Ejecución Y Evaluación De Las Pruebas De Integración

Prueba		
Caso de prueba	Imagen de la prueba	Resultado
Suite completa frontend con integración de servicios: npm run test	Evidencia: artifacts/pruebas/vitest-output-focused.txt	Test Files: 10 passed. Tests: 107 passed.
Observaciones Técnicas/funcionales		Integra servicios de los tres módulos con mocks de APIs y valida contratos de datos.
Concepto Final		Aprobado

PRUEBAS BASADAS EN HILOS

Análisis De Las Pruebas

Nombre del caso de uso	Diagrama de secuencia textual
Hilo Pacientes	Administrador -> PatientForm -> pacientesService -> pacientesApi -> API -> respuesta -> lista actualizada
Hilo Terapeutas	Administrador -> TerapeutaForm -> terapeutasService -> terapeutasApi -> API -> respuesta -> lista actualizada
Hilo Tratamientos	Administrador -> TratamientoForm -> tratamientosService -> tratamientosApi -> API -> respuesta -> lista actualizada

Tabla de estado	Valor
Inicial	Formulario vacío/listado cargando
Validando	Campos obligatorios y clases válidas/no válidas
Enviando	Servicio construye payload y llama API
Final	Registro guardado o error mostrado

Diseño De Los Casos De Pruebas

Hilo	Datos de entrada	Resultado esperado
Pacientes	Documento válido, rol 1	Paciente registrado y listado como paciente.
Terapeutas	Usuario, contraseña, licencia y rol 2	Terapeuta registrado y listado como terapeuta.
Tratamientos	Nombre, especialidad, duración, costo y equipo	Tratamiento registrado y equipo asociado.

PRUEBAS DE SISTEMAS

5.1 PRUEBAS DE RENDIMIENTO

Tipo de Prueba	RENDIMIENTO
Nombre de la Prueba	Construcción de producción de módulos Pacientes, Terapeutas y Tratamientos
Descripción de la Prueba	Verificar que la aplicación compile correctamente con los módulos incluidos.
Ambiente o condiciones previas y necesarias para su ejecución.	Node.js, npm, Vite, repositorio local.
Herramientas y metodología utilizada.	Vite build. Apache JMeter queda pendiente porque no está instalado en el equipo.
Detalle de la ejecución de la prueba (Pantallazos)	Evidencia: artifacts/pruebas/build-output.txt
Resultado de la Prueba.	Build exitoso. Vite advierte chunk JavaScript grande; se recomienda code-splitting.

5.2 PRUEBAS DE SEGURIDAD

Tipo de Prueba	SEGURIDAD
Nombre de la Prueba	Consumo de API autenticado
Descripción de la Prueba	Verificar que los módulos consuman servicios mediante apiCliente, que agrega Authorization Bearer cuando existe token.
Ambiente o condiciones previas y necesarias para su ejecución.	Frontend con sessionStorage/token disponible.
Herramientas y metodología utilizada.	Revisión de apiCliente y pruebas unitarias. OWASP ZAP queda pendiente porque no está instalado.
Detalle de la ejecución de la prueba (Pantallazos)	Evidencia: apiCliente.test.js y servicios de los tres módulos.
Resultado de la Prueba.	Se valida el manejo de token y limpieza de sesión ante 401 en la suite general.

5.3 PRUEBAS DE USABILIDAD

Tipo de Prueba	USABILIDAD
Nombre de la Prueba	Formularios de los tres módulos
Descripción de la Prueba	Verificar que los formularios tengan campos requeridos, listas, mensajes y botones de acción.
Ambiente o condiciones previas y necesarias para su ejecución.	Navegador Microsoft Edge headless y servidor Vite.
Herramientas y metodología utilizada.	Capturas de pantalla y revisión funcional.

Detalle de la ejecución de la prueba
(Pantallazos)

KineSys

Iniciar Sesión

Registrar Nuevo Paciente

×

Información Personal

Tipo Documento *

Cédula de Ciudadanía

Número Documento *

1234567890

Género

Seleccionar

Nombres *

Apellidos *

Fecha Nacimiento

dd/mm/aaaa

Email

Celular

Contacto y Ubicación

Teléfono

Ciudad

Departamento

Dirección Completa

EPS

+ Agregar EPS

Seleccionar EPS

Sin EPS

Seguro Médico

+ Agregar Seguro

Pacientes

KineSys

Iniciar Sesión

Registrar Nuevo Terapeuta

×

Información de Acceso

Usuario *

Nombre de usuario único

El nombre de usuario debe ser único en el sistema

Contraseña *

Contraseña del usuario

La contraseña debe tener al menos 6 caracteres

Información Personal

Tipo Documento *

Seleccionar

Número Documento *

1234567890

Género

Seleccionar

Nombres *

Apellidos *

Fecha Nacimiento

dd/mm/aaaa

Email

Celular

Contacto y Ubicación

Teléfono

Ciudad

Departamento

Dirección Completa

Terapeutas

KineSys

Iniciar Sesión

Registrar Nuevo Tratamiento

X

Información Básica

Nombre del Tratamiento *

Ej: Terapia de Rehabilitación

Especialidad *

Selecciona una especialidad

* Campo requerido. Selecciona la especialidad a la que pertenece este tratamiento.

Descripción

Descripción detallada del tratamiento...

Configuración de Sesiones

Duración (minutos)

30

Sesiones Recomendadas

1

Frecuencia

Semanal

Equipos Utilizados

Seleccionar Equipo

Cargando equipos...

Cantidad

1

+ Agregar

Tratamientos

Resultado de la Prueba.

Las interfaces principales cargan y muestran los campos requeridos.

5.4 PRUEBAS DE PORTABILIDAD

Tipo de Prueba	PORTABILIDAD
Nombre de la Prueba	Ejecución en ambiente local Windows
Descripción de la Prueba	Comprobar que las pruebas y el frontend se ejecuten en el ambiente local del equipo.
Ambiente o condiciones previas y necesarias para su ejecución.	Windows 11, Node.js v24.14.0, npm 11.9.0.
Herramientas y metodología utilizada.	Vitest, Jest, Vite y Edge headless.
Detalle de la ejecución de la prueba (Pantallazos)	Evidencia: ejecuciones de pruebas específicas y capturas.
Resultado de la Prueba.	Pruebas y capturas generadas correctamente en el ambiente local.

PRUEBAS DE ACEPTACIÓN

DISEÑO DE LOS CASOS DE PRUEBAS

ID Caso: CA-PAC-01	Requisito Asociado: RF-PAC-01	Escenario de Negocio
Datos de entrada: Paciente válido: Ana Pérez, CC 12345678.		Registro exitoso de paciente
Pasos de Ejecución: 1. Abrir módulo Pacientes. 2. Ingresar datos. 3. Guardar.		
Resultado Esperado: Paciente creado con rol 1.		Resultado Obtenido: Diseñado; servicio automatizado aprobado.

ID Caso: CA-TER-01	Requisito Asociado: RF-TER-01	Escenario de Negocio
Datos de entrada: Terapeuta válido con usuario, contraseña, licencia y título.		Registro exitoso de terapeuta
Pasos de Ejecución: 1. Abrir módulo Terapeutas. 2. Ingresar datos. 3. Guardar.		
Resultado Esperado: Terapeuta creado con rol 2.		Resultado Obtenido: Diseñado; servicio automatizado aprobado.

ID Caso: CA-TRA-01	Requisito Asociado: RF-TRA-01	Escenario de Negocio
Datos de entrada: Tratamiento válido con especialidad, duración, costo y frecuencia.		Registro exitoso de tratamiento
Pasos de Ejecución: 1. Abrir módulo Tratamientos. 2. Ingresar datos. 3. Guardar.		
Resultado Esperado: Tratamiento creado correctamente.		Resultado Obtenido: Diseñado; servicio automatizado aprobado.

ID Caso: CA-TRA-02	Requisito Asociado: RF-TRA-03	Escenario de Negocio
Datos de entrada: idEspecialidad = 0.		Tratamiento sin especialidad
Pasos de Ejecución: 1. Abrir tratamiento. 2. Dejar especialidad sin seleccionar. 3. Guardar.		
Resultado Esperado: Sistema muestra error y no envía datos.		Resultado Obtenido: Diseñado; validación observada en código.

EJECUCIÓN Y EVALUACIÓN DE LA PRUEBA

Prueba		
Caso de prueba	Imagen de la prueba	Resultado
Casos de aceptación de los tres módulos	Capturas incluidas en Producto del software y Pruebas de Usabilidad.	Los flujos quedan diseñados; ejecución manual completa pendiente con backend y datos reales.
Observaciones Técnicas/funcionales		La plantilla sugiere Selenium/Katalon; esas herramientas no están configuradas en el proyecto. Se deja el diseño listo para automatización posterior.
Concepto Final		Parcialmente aprobado / pendiente de ejecución E2E

CONCLUSIONES

- 1. El documento fue reorganizado siguiendo las tablas y apartados de la plantilla para los tres módulos trabajados.
- 2. El módulo Pacientes cuenta con clases de equivalencia, valores límite, camino básico, diseño de casos y 12 pruebas automatizadas aprobadas.
- 3. El módulo Terapeutas cuenta con clases de equivalencia, valores límite, camino básico, diseño de casos y 12 pruebas automatizadas aprobadas.
- 4. El módulo Tratamientos cuenta con clases de equivalencia, valores límite, camino básico, diseño de casos y 9 pruebas automatizadas aprobadas.
- 5. La suite completa frontend queda en 10 archivos y 107 pruebas aprobadas tanto en Vitest como en Jest.
- 6. Quedan pendientes pruebas E2E con Selenium/Katalon y herramientas externas JMeter/OWASP ZAP si el docente exige ejecución real con esas herramientas.